Functional decomposition

Trainer's Name: Ms. Shanmuga Priya

What is Functional Decomposition?

- Functional decomposition means breaking one large task into smaller tasks.
- Each smaller task is implemented as a separate function.
- The main program coordinates these functions.
- Each function should ideally perform one clear responsibility.

Why Do We Need Functional Decomposition?

- Large programs become difficult to understand when everything is written in one place.
- Smaller functions make the program easier to read and debug.
- Functions can be tested independently.
- The same function can be reused from different parts of the program.
- Different developers can work on different functions.

Without Functional Decomposition

```
public static void main(String[] args) {
    // display menu
    // read choice
    // create account
    // deposit money
    // withdraw money
    // check balance
    // validate input
    // display results
    // ... lots of code here
}
```

Problem: main() becomes long, difficult to read, test and maintain, if we write all the code inside main function

With Functional Decomposition

```
public static void main(String[] args) {
    displayMenu();
    int choice = readChoice();

    if (choice == 1)
        createAccount();
    else if (choice == 2)
        depositMoney();
    else if (choice == 3)
        withdrawMoney();
    else if (choice == 4)
        checkBalance();
}
```

main() now focuses on the overall flow; details are handled by functions which is defined in another class.

Breaking the Bank Program into Functions

displayMenu() → shows available banking operations

readChoice() → gets the user's choice

createAccount() → creates an account

depositMoney() → adds money to the account

withdrawMoney() → removes money after validation

checkBalance() → displays the current balance

Java Example – Bank Class

```
class Bank {
    double balance = 1000;
}
class BankService
{
    void deposit(double amount) {
        balance = balance + amount;
    }

    void withdraw(double amount) {
        if (amount <= balance)
            balance = balance - amount;
        else
            System.out.println("Insufficient balance");
    }

    double getBalance() {
        return balance;
    }
}
```

Note :
Since in realtime, we will not write the logic in the Entity class, its better to create another
class called BankService, in that we can define the methods

Java Example – BankMain Class

```
class BankMain {
    public static void main(String[] args) {
        Bank bank = new Bank();

        bank.deposit(500);
        System.out.println("Balance: " +
bank.getBalance());

        bank.withdraw(200);
        System.out.println("Balance: " +
bank.getBalance());
    }
}
```

A Function Can Also Return Data

```
double balance = bank.getBalance();

if (balance >= 1000) {
    System.out.println("Minimum balance maintained");
} else {
    System.out.println("Maintain minimum balance");
}
```

getBalance() performs one responsibility: returning the balance.

The returned value can then be used for further processing.

Benefits of Functional Decomposition

Readability – code is easier to understand.

Maintainability – changes can be made in smaller sections.

Reusability – functions can be called multiple times.

Testing – each function can be tested separately.

Debugging – errors are easier to locate.

Teamwork – responsibilities can be divided among developers.

Key Takeaways

Functional decomposition = break a large problem into smaller functions.

Each function should have a clear responsibility.

main() should coordinate the overall flow instead of containing every detail.

Functions may take input, process it, and return output.

Good decomposition makes programs easier to read, test and maintain.

THANK YOU